

Contributor Guide

Table of contents

GeckoView Contributor Quick Start Guide

This is a guide for developers who want to contribute to the GeckoView project. If you want to get started using GeckoView in your app then you should refer to the [wiki](#).

Background

GeckoView is a public API that exposes core Gecko functionality to GeckoView consumers. Several Mozilla products use GeckoView as their core entry-way into Gecko. For example, Android Components has GeckoView as a dependency and uses it to communicate with Gecko. Fenix, Focus, and Reference Browser have Android Components as a dependency and build a browser on top of this framework.

The architecture, in broad strokes, looks like this:

Gecko <-> GeckoView <-> Android Components <-> Fenix or Focus or Reference Browser

- **Gecko** is the fundamental platform that the rest are built on. It is a multilayered robust platform that conforms to standards related to browser mechanics.
- **GeckoView** exposes key portions of Gecko for API consumers to use in Android systems. A subset of key Gecko functionality is exposed this way. It is a public API, so public API changes are designed to always be non-breaking and follow a deprecation process.
- **Android Components** links to GeckoView to support Gecko browsers. It is platform-independent of Gecko and GeckoView. For example, another browser engine could be used instead of Gecko. It also provides other reusable browser-building components.

- **Fenix or Focus or Reference Browser** are the end app products. They contain the primary view layer and connect everything together to form a final product.

Please keep this architecture in mind while solving bugs. Identifying the correct layer to solve a bug is an important first step for any bug.

Performing a bug fix

As a first step, you need to set up [mozilla-central](#), and [Bootstrap](#) and build the project.

Once you have got GeckoView building and running, you will want to start contributing. The [Firefox Contributors' Quick Reference](#) covers writing, submitting and updating a patch. To contribute to GeckoView specifically, you will need the following additional information.

Debugging code

Because GeckoView is on many layers, often the best debugging tool depends on the layer the bug is on. For Java or Kotlin code, using the Android Studio IDE is an easy way to connect a debugger and set breakpoints. It can also be used with C++ code once native debugging is setup. Please see this guide on [Native Debugging](#).

For JavaScript code, it is possible to connect a debugger using Firefox Desktop Nightly's `about:debugging` section. The device must be setup to support USB connections and the device likely needs developer mode enabled.

Sometimes it is easier to leave logs to help trace exactly which layer the bug is on first, to do this on the various layers:

- **Java** - `Log.i("Any Tag", "Any string " + variable.toString())`
 - May be used for permanent logging in GeckoView, but should follow existing logging practices. Requires an import.
 - Note, Android Components has different permanent logging practices.
- **JavaScript** - `dump("Any string " + variable)`
 - May not be used for permanent logging.

- May need to use `JSON.stringify(variable)` to deserialize objects in JavaScript.
- **JavaScript** - `debug`Any String ${variable}``
 - May be used for permanent logging, but should follow existing logging practices. Requires an import.
 - Recommend `dump` for earlier debugging.
- **JavaScript** - `console.log("Any String " + variable)`
 - May be viewed using `about:debugging` on Desktop to a connected device and then connecting to the content process.
- **C++** - `MOZ_LOG`
 - Use `about:logging` or set the `logging` preference directly. For example, `--setpref="logging.LOG_MODULE_NAME=LOG_MODULE_VALUE"` or manually in `about:config`.
- **C++** - `printf_stderr("Any String")` or `__android_log_write(ANDROID_LOG_INFO, "Any Tag", "Any string");` or `__android_log_print(ANDROID_LOG_INFO, "Any Tag", "Any string");`
 - None of these may be used for permanent logging, only for debugging. Use `MOZ_LOG` for permanent logging.
 - Variable logging will need to be converted to a C string or other supported logging string.
 - Permanent logging for a GeckoView C++ file could be setup similar to:

```
#define GVS_LOG(...) MOZ_LOG(sGVSupportLog, LogLevel::Info,
(__VA_ARGS__))
static mozilla::LazyLogModule sGVSupportLog("Any Tag");
GVS_LOG("Any string");
```

Please be sure to remove any non-permanent debugging logs prior to requesting landing.

To view logging on Android, attach a device, and run `adb logcat -v color` for colorful logs or else use the Android Studio log frame.

Running tests and linter locally

To ensure that your patch does not break existing functionality in GeckoView, you can run the junit test suite with the following command

```
./mach geckoview-junit
```

This command also allows you to run individual tests or test classes, e.g.

```
./mach geckoview-junit org.mozilla.geckoview.test.NavigationDelegateTest
./mach geckoview-junit
org.mozilla.geckoview.test.NavigationDelegateTest#LoadUnknownHost
```

To see information on other options, simply run `./mach geckoview-junit --help`; of particular note for dealing with intermittent test failures are `--repeat N` and `--run-until-failure`, both of which do exactly what you'd expect. If a test is intermittently failing, consult [Debugging Intermittent Test Failures](#) for additional tips.

Other tests, such as mochitests, may be ran using:

```
./mach test <path-or-dir-to-test>
```

Core GeckoView lints are:

```
# Will perform general Android specific formatting and linting.
./mach lint -l android-format
# Will determine if GeckoView API changes happened, find more info at API
documentation below, if changes occurred.
./mach lint --linter android-api-lint
# Will perform static analysis and report required changes.
./mach lint --warnings --outgoing
```

For the linters below, add `--fix` to the command for the linter to fix the issue automatically. Note, using `--fix` will make changes. Most commands also accept a specific file or directory to speed up linting.

If your patch makes a GeckoView JavaScript module, you should run ESLint:

```
./mach lint -l eslint mobile/android/modules/geckoview/
```

If your patch makes a C++ file change, you should run the C++ linter formatter:

```
./mach format <path/to/file.cpp>
```

If your patch makes a Python file change:

```
./mach lint --linter flake8  
./mach lint --linter black
```

Additionally, sometimes lints can be automatically detected and fixed on certain files, for example:

```
# Will attempt to detect the linter and fix any issues.  
# Note, using ``--fix`` will make code changes.  
./mach lint --fix <path-to-file>
```

Updating the changelog and API documentation

GeckoView is a public API. We carefully document changes so that other users of the API can update accordingly.

If the patch has made public API changes, then a new `api.txt` and `CHANGELOG.md` entry will be needed. Additionally, a hash is generated to track these changes. This hash, which is also sometimes referred to as an API key in this document, is added to the bottom of the changelog.

The following steps will go through the commands of checking for public API changes, generating a new updated `api.txt`, generating a new API hash, and recommendations for a `CHANGELOG.md` entry.

To check whether your patch has altered the API, run the following command:

```
./mach lint --linter android-api-lint
```

The output of this command will inform you if any changes you have made break the existing API. The first run of the command will tell you if there are API changes. It will also provide the next command. (A second run at the end of the `api.txt` generating process will state the API hash.)

If a change is needed, the next command to preview the expected API changes will be:

```
./mach gradle geckoview:apiLintDebug
```

Running the above command should cause the build to fail and will explain what it detected. The next step will generate a new `api.txt` file, which is a running list of public functions.

If the changes look as expected, then it is time to generate `api.txt` by running this command:

```
./mach gradle geckoview:apiUpdateFileDebug
```

The build should pass this time, and an updated `api.txt` file will be generated for the changes.

Finally, follow the next command to get the API hash for the CHANGELOG:

```
./mach lint --linter android-api-lint
```

In the `CHANGELOG` file, at the bottom, swap `[api-version]` with the new hash provided by the linter and add an entry for what changed.

The feature and methods should also be documented in the changelog similar to the example below.

The format should be as follows:

```
- Summary of changes that should mention the method name, along with the
respective class /
    interface name, the major version and the index, and the bug ID, along
with the
    bugzilla link.

[<major_version>.<index>]: {{javadoc_uri}}/<url_path>
```

To determine the index, take the next index in the list of `[<major_version>.<index>]`. If no list is present, start with `index = 1`.

- **Example for Adding a Method Changelog Entry**

```
# v71
- Added [`GeckoRuntimeSettings.Builder#aboutConfigEnabled`][71.12] to
control whether or
    not `about:config` should be available.
    ([bug 1540065]({{bugzilla}}1540065))

[71.12]:
{{javadoc_uri}}/GeckoRuntimeSettings.Builder.html#aboutConfigEnabled(boolean)
```

On more complicated features, sometimes it isn't obvious what the URL signature is for the CHANGELOG. In that case, create the javadoc locally (instructions below) to confirm the URL paths.

If patch changes are made during review, be sure to re-generate new signatures and API hashes. Generally, unstaging `api.txt` is the easiest way to accomplish this.

A special situation is when a patch changing the API may need to be uplifted to an earlier branch of mozilla-central, for example, to the beta channel. To do this, follow the usual uplift steps and make a version of the patch for uplift that is graphed onto the new target branch and rerun the API linter commands and update the CHANGELOG to the release it will be graphed onto.

Creating JavaDoc Locally

GeckoView is a public API, so well maintained javadoc is an important practice. To create the javadoc locally, we use the following command:

```
./mach android geckoview-docs
```

To view the javadoc locally, first find the directory:

- In your `mozilla-unified` directory, type the following command:

```
find . -name javadoc -path '*/geckoview/*'
```

This should return the relative path of the local geckoview javadoc.

As an example, the output could be this:

```
./obj-aarch64-unknown-linux-  
android/gradle/build/mobile/android/geckoview/docs/javadoc
```

Then, use the following command to go into the directory of the local javadoc:

```
cd path_of_javadoc_from_above
```

Now, we want to launch a local web server. To launch locally, use any web server, for example:

```
python3 -m http.server 8000
```

In this example, navigate to the web docs via `http://localhost:8000/`.

Note

If you get a 404 error, please ensure that you have navigated to the correct directory and try launching the web server again.

Then, look for the changed method in the list displayed on the webpage and click into it.

For using this to identify URLs for the `CHANGELOG`, copy everything after `.../org/mozilla/geckoview/`. Fill in the entry by doing `{{javadoc_uri}}/paste_the_copied_text`. See the example above for reference.

Deprecation Policy

GeckoView follows a deprecation policy you can learn more in this [design doc](#).

To deprecate an API, add the deprecation flags with an identifier for a deprecation notice, so that all notices with the same identifier will be removed at the same time (see below for an example). The version is the major version of when we expect to remove the deprecated member attached to the annotation. The GeckoView team instituted a deprecation policy which requires each backward-incompatible change to keep the old code for 3 releases, allowing downstream consumers, like Fenix, time to migrate asynchronously to the new code without breaking the build.

```
@Deprecated
@DeprecationSchedule(id = "<interface_or_class_of_method>-<method_name>",
    version = <Current Nightly + 3>)
```

Since this is a public API, the changelog must also be updated. Please ensure that you follow the correct format for changelog entries. Under the heading for the next release version, add a new entry for the changes that you are making to the API, along with links to any relevant files, and bug number.

- **Example for Deprecating a Method Changelog**

```
- ⚠️ Deprecated [GeckoSession.ContentDelegate.onProductUrl`][128.5], will
  now be removed in v131.
  ([bug 1898055]({{bugzilla}}1898055))

[128.5]:
  {{javadoc_uri}}/GeckoSession.ContentDelegate.html#onProductUrl(org.mozilla.gecko
```

If an API is deprecated, file a follow-up bug or leave the bug open by adding the keyword `leave-open` to remove and clean up the deprecated API for the version it is to be removed on. Also, ensure that running the API linter commands has changed the javadoc of the deprecated method to indicate that the method has been scheduled for deprecation. If not, ensure to do this manually.

Submitting to the `try` server

It is advisable to run your tests before submitting your patch. You can do this using Mozilla's `try` server. To submit a GeckoView patch to `try` before submitting it for review, type:

```
./mach try --preset android-geckoview
```

This will automatically run critical tests from the GeckoView test suite. If your patch passes on `try` you can be (fairly) confident that it will land successfully after review.

Failures on `try` will show up with the test name highlighted in orange. Select the test to find out more. Intermittent failures occasionally occur due to issues with the test harness. Retriggering the test is a good way to confirm it is an intermittent failure and not due to the patch. Usually there will also be a bug number with a portion of the stack trace as well for documented intermittent failures. See [Intermittent Test Failures](#) for more information.

To debug failures on try, it is always a good idea to check the logcat. To do this, select the individual test, select “Artifacts and Debugging” and then open the log from “logcat-emulator-5554.log”.

Tagging a reviewer

When submitting a patch to Phabricator, if you know who you want to review your patch, put their Phabricator handle against the `reviewers` field.

If you don't know who to tag for a review in the Phabricator submission message, leave the field blank and, after submission, follow the link to the patch in Phabricator and scroll to the bottom of the screen until you see

the comment box.

- Select the `Add Action` drop down and pick the `Change Reviewers` option.
- In the presented box, add `geckoview-reviewers`. Selecting this group as the reviewer will notify all the members of the GeckoView team there is a patch to review.
- Click `Submit` to submit the reviewer change request.

GeckoView, Android Components, Fenix, Focus, and Reference Browser Dependency Substitution

Internal product dependency substitution is handled automatically in mozilla-central on full builds. When building, the substitution into these other products will happen automatically after `./mach build` is ran. However, in artifact builds, changes in Gecko or GeckoView will not consistently be reflected. If making changes to Gecko or GeckoView, it is **strongly** recommended to only use full builds as changes in Gecko or GeckoView may not be reflected when using artifact builds.

Include GeckoView as a dependency

If you want to include a development version of GeckoView as a dependency inside another app, you must link to a local copy. There are several ways to achieve this, but the preferred way is to use Gradle's *dependency substitution* mechanism, for which there is first-class support in `mozilla-central` and a pattern throughout Mozilla's GeckoView-consuming ecosystem.

The good news is that `mach build` produces everything you need, so that after the configuration below, you should find that the following commands rebuild your local GeckoView and then consume your local version in the downstream project.

```
cd /path/to/mozilla-central && ./mach build
cd /path/to/project && ./gradlew assembleDebug
```

Be sure that your `mozconfig` specifies the correct `--target` argument for your target device. Many projects use “ABI splitting” to include only the target device's native code libraries in APKs deployed to

the device. On x86-64 and aarch64 devices, this can result in GeckoView failing to find any libraries, because valid x86 and ARM libraries were not included in a deployed APK. Avoid this by setting `--target` to the exact ABI that your device supports.

Dependency substituting your local GeckoView into a non-Mozilla project

In projects that don't have first-class support for dependency substitution already, you can do the substitution yourself. See the documentation in [substitute-local-geckoview.gradle](#), but roughly: in each Gradle project that consumes GeckoView, i.e., in each `build.gradle` with a `dependencies { ... 'org.mozilla.geckoview:geckoview-...' }` block, include lines like:

```
ext.topsrcdir = "/path/to/mozilla-central"
ext.topobjdir = "/path/to/object-directory" // Optional.
apply from: "${topsrcdir}/substitute-local-geckoview.gradle"
```

Remember to remove the lines from all `build.gradle` files when you want to return to using the published GeckoView builds!

Next Steps

- Get started with [Native Debugging for Android](#)